

UI/UX testing

First Impression & Clarity (Landing UX)

HI: Analyze this UI (screens/components) from a first-time user perspective.

Focus on:

- What the product appears to do within 5–10 seconds
- The first obvious action a user would take
- Any confusion in terminology or layout

Output format:

1. What this product seems to do (in one sentence)
2. First action a user is likely to take
3. Points of confusion or ambiguity
4. Whether the value proposition is clear within 30 seconds

Be brutally honest. Assume no prior context.

AI:

1. What This Product Seems to Do (One Sentence)

JoaLLM is a multi-tool AI workspace where you can chat with LLMs, upload documents to query with RAG, build visual automation workflows, and run interactive notebooks — all from a single dashboard.

That said, a first-time visitor landing on the platform (not the marketing site) would likely need 15–20 seconds to piece that together, not 5–10.

2. First Action a User Is Likely to Take

On the landing page: click "Launch Platform →" — the CTA is prominent and repeated.

After login, on the Welcome page: click "Start Chatting" — it's the first button above the fold, familiar to anyone who's used ChatGPT.

Most users will default to Chat and never discover the other 5 features without deliberate exploration.

3. Points of Confusion or Ambiguity

Terminology friction:

- "JoaLLM Farm" — completely opaque. A new user has no idea this is a model browser/selector. "Farm" suggests compute infrastructure, not a catalogue. It should be "Model Library" or "AI Models."
- "RAG Search" / "Knowledge Base" — used interchangeably in copy, sidebar, and navigation. The sidebar says "Knowledge Base - Chat, Search & Manage Documents" but the route is /rag-search. Internal jargon ("RAG") is exposed to users who may not know the acronym.
- "LLM Hub" in the sidebar header — this is a section label for the nav, but it reads like a feature. Confusing alongside "JoaLLM Farm" which also sounds hub-like.
- "Documentation - Work in Progress & Planning" — this card is literally in the feature grid on the welcome screen. Shipping an unfinished item as a primary navigation card erodes trust immediately.

Layout/hierarchy issues:

- The welcome page has 6 feature cards + 6 capability cards + 4 quick prompts + a CTA section — this is a wall of cards. There's no clear visual hierarchy telling the user "start here."
- The sidebar has both navigation AND quick-start prompts AND quick actions — three different intent levels competing for attention in one panel.
- The header has 5 icon buttons with no labels (chat, RAG, notebook, settings, knowledge manager). New users won't know what each icon does without hovering for tooltips. On mobile this is invisible entirely.
- Model selector is hidden on mobile — but model selection is a core differentiator of the product. Hiding it contradicts the product's positioning.

Copy inconsistencies:

- Landing page says "GPT-4 Turbo, Claude 3" but the tech stack section elsewhere references GPT-4o and Claude 4. The feature cards on the welcome screen are outdated.
- "Notebook" lists "Python Runtime" as a benefit in the feature card, but the actual implementation uses JS/TS sandbox. This is a mismatch that will frustrate developers.

- "35+ File Formats" appears prominently as a stat, but there's no visible list or confirmation of this anywhere in-product.

4. Is the Value Proposition Clear Within 30 Seconds?

On the landing page: Yes, roughly.

The headline "Upload. Query. Automate. Ship AI Products Faster." is direct. The feature grid with icons and technical tags communicates depth quickly. Technical users will get it in ~15 seconds.

On the platform itself (after login): No.

The welcome screen tries to be a product tour, a feature showcase, and a quick-start guide simultaneously. The headline "Your AI Command Center" is generic — it could describe any AI tool. There's nothing that immediately distinguishes this from ChatGPT + Notion + Zapier used separately.

The core differentiator — native pgvector RAG + multi-LLM + workflows in one product — is buried in paragraph text on feature cards. A new user focused on "Start Chatting" would miss it entirely.

Summary Verdict

Dimension	Score
First impression clarity	6/10 – landing page headline works, platform welcome page doesn't
Navigation intuitiveness	5/10 – "Farm," "Hub," unlabeled icons create friction
Value prop legibility in 30s	6/10 – good on landing, weak in-app
Onboarding guidance	4/10 – no actual onboarding flow; just a feature grid
Trust signals	7/10 – tech stack detail and security copy build credibility

Biggest single fix: Replace the welcome screen's card wall with a linear 3-step onboarding ("Upload a doc → Ask it a question → See the result") that demonstrates the RAG differentiator in 60 seconds. That's what separates JoaLLM from just another ChatGPT wrapper — but users never see it unless they go looking.

Onboarding & Time-to-Value

HI: Evaluate how a new user would go from signup/login to first meaningful output.

Focus on:

- Number of steps required
- Clarity of next actions
- Presence of guidance (tooltips, prompts, defaults)

Output format:

1. Step-by-step onboarding journey
2. Estimated time to first value (in minutes)
3. Friction points
4. Suggestions to reduce steps by 50%

Define "value" as: getting a useful AI output from uploaded knowledge.

Codebase: 1. Step-by-Step Onboarding Journey

Goal: Signup → Upload doc → Get RAG answer

#	Step	Screen	Required Input	Notes
1	Land on site	Landing page	-	CTA: "Launch Platform -"
2	Click "Launch Platform"	→ /register	-	
3	Fill registration form	RegisterPage	Email, password (8+ chars), name (2+ chars)	3 separate fields, no hints
4	Submit → auto-login	WelcomePage	-	No email verification, goes straight in
5	Parse the welcome screen	WelcomePage	Decision	6 feature cards, no "start here" cue
6	Find Knowledge Base	WelcomePage	Click "Knowledge Base" card	Not the most prominent option
7	Land on RAG Search page	/rag-search	-	3 tabs: Chat / Search / Knowledge Manager
8	Find upload	Click "Knowledge Manager" tab	Tab navigation	Upload is on the 3rd tab, not visible first
9	Upload document	KnowledgeManagerNew	Select/drag file	10 MB max, PDF marked "Beta"
10	Wait for processing	Same page	-	2-10 sec; status badge updates
11	Navigate to RAG chat	Click "Chat with Knowledge Base" tab	Tab navigation	Must go back to the 1st tab
12	Type a question	RAGChatInterface	Message text	No pre-filled examples visible here
13	Read response + sources	Same page	-	Sources shown with relevance %

Total: 13 steps, 4 of which are navigation-only (no user input, just clicks)

2. Estimated Time to First Value

Phase	Time
Registration form	~45 sec
Parsing welcome screen + finding Knowledge Base	~30 sec
Finding the upload tab (3rd tab)	~15 sec
File selection + upload	~20 sec
Processing (backend)	2–10 sec
Navigating back to Chat tab	~10 sec
Typing first question	~20 sec
Waiting for RAG response	~3 sec
Total	~2.5–3 minutes

For a non-technical user unfamiliar with "RAG" or "Knowledge Manager" terminology, realistically 4–5 minutes — if they find the right tab at all. There's no prompt, tooltip, or empty state that says "upload a doc first, then chat with it."

3. Friction Points

Structural friction (flow-breaking):

- Upload is on tab 3, chat is on tab 1. A user lands on the RAG page and sees a chat interface with no documents. There's no message saying "upload a document first." They'll type a question, get a vacuous or empty response, and not understand why.
- No empty-state CTA in the chat tab. When 0 documents are indexed, the chat interface still shows the input box. It should be replaced with "Upload a document to get started →."
- "Knowledge Manager" tab label doesn't say "Upload" — users scan for the word "upload," not "manage." The tab is mislabeled for its primary action.

Cognitive friction (requires interpretation):

- Welcome page has no default path. 6 equal-weight feature cards means the user must decide what to do. First-time users need a default, not options.
- "RAG Chat" terminology is exposed at every level — tab labels, headings, API routes. Users don't think "I want RAG," they think "I want to ask my document a question."
- Model selector is present but unexplained. Default is llama-3.1-8b-instant — a name that means nothing to a new user. No label explains what it affects.
- PDF is the most common upload format but is marked "Beta (limited extraction)." This will trip up most users on their first try.

Trust friction:

- "Documentation — Work in Progress & Planning" is a nav card in the main feature grid. This signals the product isn't finished and creates hesitation before the user has even started.

- No confirmation or guidance after upload. The toast says "Document is being indexed..." but there's no follow-up prompt to go ask a question. The user is left to figure out the next step.

4. How to Cut Steps by 50% (13 → 6)

The core insight: the happy path should be a single linear screen, not a navigation puzzle.

Proposed 6-step flow:

#	Step	Change from current
1	Register (email + password only)	Drop "name" field – collect later in settings
2	Land on a purpose-built "Get Started" page	Replace the welcome card grid with one screen: "Upload a document, then ask it anything"
3	Upload file inline on that page	Drag-drop zone is the visual center, not buried in tab 3
4	See "Ready – ask your first question" state	Inline status, same page. No navigation required.
5	Type question into input that appears after processing	Input is locked/grayed until doc is ready, then unlocks with animation
6	Read RAG response with sources	Same page

Specific implementation changes:

1. Collapse the RAG page tabs into a single view. When 0 docs: show upload zone + "or try these sample questions →" (preloaded demo docs). When docs exist: show upload zone (collapsed/minimized) + chat input. No tab navigation.

2. Add an empty-state intercept in the chat tab. If documentsCount === 0, replace the message input with: "No documents yet. Upload one to chat with your knowledge base. [Upload a file →]"

3. Make the welcome screen redirect automatically for new users (check usageStats.totalFiles === 0 && totalRequests === 0). Skip the card grid entirely on first login.

4. Replace "Knowledge Manager" tab label with "Upload." One word. No ambiguity.

5. Pre-fill a suggested question after upload completes, based on the filename: "What is [filename] about?" — one click to submit.

6. Show a persistent "First time? Start here →" banner that deep-links to the upload flow, visible until the user has had their first RAG conversation. Dismiss on completion.

Bottom line: The core happy path (upload → query) works and is fast on the backend (2–10 sec processing). The friction is entirely in the UI navigation — users are being asked to find a 3-tab interface inside a 6-card feature grid to accomplish what should be a single-page, guided experience. The value is real; the path to it is unnecessarily hidden.

Knowledge Upload + RAG UX (CORE FLOW)

HI:Analyze the document upload and retrieval experience.

Focus on:

- How users upload documents
- Feedback after upload (status, readiness)
- Query experience and response clarity
- Visibility into source grounding

Output format:

1. Upload flow steps
2. How system communicates “ready to query”
3. Query → response experience
4. Whether user can trust the output
5. Missing feedback or transparency elements

Prioritize trust and clarity over visual design.

1. Upload Flow Steps

Exact sequence a user experiences:

#	Action	What user sees	What actually happens
1	Navigate to Knowledge Manager tab	Drag-drop zone + dashed border	Nothing – blank until interaction
2	Drop/select file	Border highlights red, scale animation	Client-side validation (type, size)
3	File appears in queue	Filename + size + progress bar	HTTP multipart upload begins
4	Upload completes	Progress bar fills → disappears	Server writes file to disk, creates DB record (status='uploaded')
5	Processing begins	Spinner + "Processing..." text below filename	Backend: extract text → chunk → embed → pgvector insert
6	Doc list updates (every 5s poll)	Status badge changes: yellow "Processing" → green "Indexed & Searchable"	status='processed' written to DB
7	Toast appears	"[filename] is now available for semantic search!"	Frontend sees status change in poll response

Key detail: The progress bar tracks the HTTP upload only — not processing. After the bar fills at 100%, there's a separate "Processing..." spinner with no progress indication. To a user these look like two disconnected phases, not one pipeline.

2. How the System Communicates "Ready to Query"

There are three separate signals for readiness — none of them are coordinated:

Signal 1 — Toast notification:

█ "[filename] is now available for semantic search!"

Appears once, auto-dismisses. If the user has scrolled or switched tabs, it's gone.

Signal 2 — Status badge on document row:

- uploaded → blue badge
- processing → yellow badge + animated spinner
- processed → green badge "Indexed & Searchable"
- failed → red badge + error message text

This is the most durable signal, but it's on the Knowledge Manager tab (tab 3). The user must be looking at the right tab to see it.

Signal 3 — Document count in chat tab:

┆ "X docs" pill next to "Suggested questions based on your documents"

This counter updates when the user switches to the Chat tab and the poll fires. There's no push notification — it's passive.

What's missing: There's no signal that bridges the two tabs. A user who uploads on tab 3 and immediately switches to tab 1 (chat) sees no indication that processing is still happening and their document isn't ready yet. They can type a question and get an empty result with no explanation.

3. Query → Response Experience

When documents exist and query matches:

User types a question → hits Enter → sees:

1. Their message in a right-aligned bubble
2. A typing/loading indicator (implied by streaming)
3. AI response in left-aligned bubble with markdown rendering
4. Below the response: a "Knowledge Sources" panel with:
 - "Based on N document(s)" subtitle
 - Up to 3 source cards, each showing:
 - Numbered badge (1, 2, 3)
 - Filename
 - Green dot + "XX% relevant" score (score × 100, rounded)
 - First 120 characters of the chunk in a bordered quote box
 - If >3 sources: "+N more sources available" pill (but no way to expand/view them)

When query matches nothing (score < 0.7 threshold):

The model returns this exact hardcoded string:

┆ "I couldn't find relevant information in the knowledge base to answer your question.
┆ Please try:
┆ • Rephrasing your question with different keywords
┆ • Checking if the relevant documents are uploaded and processed
┆ • Browsing the document list to see what's available in the knowledge base"

This response is returned as if it came from the AI model — same styling, same bubble. A user cannot distinguish "AI said it found nothing" from "the system found nothing and is scripting the AI's response." The "confidence": "none" field exists in the API response but is never displayed to the user.

Empty state (no documents uploaded yet):

The chat tab shows suggested questions with a loading skeleton, then populates with AI-generated suggestions based on uploaded documents. When there are 0 documents, suggestions still render (from a fallback set), which makes the interface look functional when it isn't. The user can click a suggested question and ask it — and will get a "no relevant information" response.

There's nothing blocking the input or indicating a prerequisite step.

4. Whether the User Can Trust the Output

What builds trust:

- Source cards with filenames are concrete — user can see which document was used
- Relevance percentage (e.g., "87% relevant") gives a sense of confidence
- Numbered sources with chunk previews let a moderately technical user cross-reference
- 0.7 relevance threshold is enforced server-side — low-quality matches are filtered

What undermines trust:

Problem 1 — Score is misrepresented.

The score is cosine distance converted to a percentage label: $(\text{score} * 100).toFixed(0) + \%$ relevant". Cosine similarity of 0.71 becomes "71% relevant" — but what does 71% mean? Is that good or bad? There's no baseline, no legend, no explanation. Users will either over-trust a 71% or dismiss an 88% because they don't know the scale.

Problem 2 — Chunks are truncated to 120 characters.

The quoted excerpt in each source card cuts off at 120 characters. This is enough to look like evidence but not enough to verify it. A user cannot read the full chunk that the model used. There's no "expand" button, no way to open the source document, no page number.

Problem 3 — "+N more sources available" goes nowhere.

When more than 3 sources are retrieved, a pill shows "+2 more sources available" but clicking it does nothing — there's no expanded view, modal, or panel. The user knows there's more evidence but can't access it.

Problem 4 — The "no results" response is indistinguishable from a model reply.

When retrieval returns 0 results, the fallback text appears in the AI message bubble with no visual differentiation. A user might read it as the AI's analysis ("I checked and found nothing") rather than the system's failure state ("retrieval returned 0 chunks"). These are fundamentally different situations.

Problem 5 — PDF is marked "Beta" but is the dominant upload format.

The upload zone says "PDF (Beta)" in small gray text. PDFs are likely 80%+ of what users want to upload. "Beta text extraction may be incomplete" is mentioned in the toast, but the user has no way to know what was actually extracted. There's no preview of the extracted text, no chunk count visible until after processing, and no indication of which pages were parsed vs. skipped.

Problem 6 — No conversation-level source tracking.

Sources are shown per-message. If a user asks 5 questions in a conversation, they get 5 separate source panels. There's no consolidated view of "all documents this conversation drew from." In a long chat, earlier source panels scroll out of view and are effectively lost.

5. Missing Feedback and Transparency Elements

Ranked by user trust impact:

Critical — not present:

Critical — not present:

Missing element	Where it's needed	Why it matters
Full chunk viewer	Source card	User can't verify what the model actually read
"Retrieval failed" vs "AI response" visual distinction	No-results state	User can't tell system failure from model answer
Extracted text preview	Post-upload	User can't confirm PDF was parsed correctly
Processing progress for the embedding phase	Upload flow	After bar fills, user sees a spinner with no ETA or stage label
Cross-tab readiness signal	Chat tab	User switches to chat while doc is still processing; no indication

High — present but inadequate:

Element	Current state	What's needed
Relevance score	Raw score * 100 percentage	Label: "Strong / Moderate / Weak match" + tooltip explaining scale
Chunk preview	120-char truncation	Expandable full-chunk view
Extra sources	"+N more" pill (non-interactive)	Collapsible list or modal
confidence: "none" field	Returned by API, never displayed	Show a "No matching content found" badge on the response
Chunk count per document	Not shown during/after upload	"47 chunks indexed" confirms how much of the doc was captured

Medium — quality-of-life:

Element	Impact
Page number in source metadata	Backend stores pageNumber in chunk metadata but UI never renders it
Source document link	No way to open the original file from a citation
"Last queried" or "used in N conversations" per document	Helps user know which docs are actually being referenced
Confidence calibration explanation	One-time tooltip: "Scores above 80% indicate strong match; below 70% are filtered out"

Bottom line: The retrieval mechanics are solid — hybrid search, 0.7 threshold, Cohere embeddings, proper pgvector indexing. The trust gap is entirely in the presentation layer. The system knows its confidence level, which chunks it used, how many pages it processed, and whether retrieval returned results — but almost none of that information reaches the user in a readable, actionable form. The backend is transparent; the UI is opaque.

Notebook UX Evaluation

Evaluate the notebook interface from a usability perspective.

Focus on:

- Clarity of different cell types
- Ease of running and iterating workflows
- State persistence and context handling
- Cognitive load for new users

Output format:

1. What users can do in the notebook (practically)
2. Learning curve (low/medium/high with reasoning)
3. Points of confusion
4. Comparison to Jupyter / ChatGPT (simpler or harder?)
5. Recommendations to simplify usage

1. What Users Can Actually Do in the Notebook

Cell Type	What actually works	What doesn't
Markdown (Text)	Write formatted notes, headings, prose; renders on deselect	Nothing missing
Code	Run JavaScript in a server-side sandbox; captures console.log output; 5s timeout	No variable scope across cells; isolated per execution
AI	Send a prompt to claude-haiku-4.5-20251001; get a text response	No awareness of other cells' content or outputs
Knowledge	Upload files per-cell, run RAG search against them	Search only; cannot pass retrieved content to adjacent AI cells automatically
Chart	Type a JS config comment	Nothing renders – chart execution is a default switch case that returns the cell type name
Agent	Fill in a name, role, and check tool boxes (6 fake tools)	Nothing executes – backend falls through to default case
Debug	Click "Start Trace", select a debug target from a dropdown	Console shows "🐞 Debug Console Ready" / "Waiting for execution trace..." permanently – not wired to anything

Practical reality: 4 of 7 cell types are functional (Markdown, Code, AI, Knowledge). Chart, Agent, and Debug are UI-only placeholders with no execution backend. A user clicking through all 7 types will discover three dead ends with no indication that they're incomplete.

2. Learning Curve: High

Reasoning:

The interface presents itself as a general-purpose notebook — visually similar to Jupyter — implying a mental model where cells are connected, stateful, and composable. None of that is true here.

- No shared variable scope. A Code cell cannot read a variable defined in the previous Code cell. Each execution is independent. This is the foundational assumption of every notebook tool the user has ever used, and it's absent.
- No inter-cell context. An AI cell has no knowledge of what other cells contain or produced. Pasting output from one cell into another is manual copy-paste.
- Cell type selection is front-loaded. You must choose a type when adding a cell. There's no way to change a cell's type after creation. If you pick "Agent" and discover it does nothing, you delete the cell and start over — but there's no delete button.

- No delete button. executionCount is tracked in the DB. Cell deletion isn't in the UI. The only way to remove a cell is presumably via the API directly or a workaround not exposed in the interface.

- The learning curve is steep not because the tool is powerful, but because its gaps are invisible. The interface looks complete. Chart cells look like they should render charts. Agent cells look like they configure agents. A new user will spend 20–30 minutes trying to get these features to work before concluding they're broken.

3. Points of Confusion

Structural confusion:

- "Text" vs "Markdown" — the button at the bottom says "Text," the cell type is markdown, the initial content is # New Section\n\nEdit this cell.... A non-technical user doesn't know what markdown is; a technical user expects the button to say "Markdown."

- "AI" cell uses BarChart3 icon — the AI cell type's icon is a bar chart. The Chart cell type also uses BarChart3. Two different cell types share an icon, and the icons don't match the cell's function. This is almost certainly a copy-paste error.

- Knowledge cell is per-cell, not notebook-scoped. Each Knowledge cell has its own attached documents. But a user building a multi-cell notebook would expect uploaded files to be available everywhere — especially in AI cells. If you upload a PDF to a Knowledge cell and then ask an AI cell a question about it, the AI cell has no access to that document.

- "Add cell below" plus button is inert. The + icon appears on hover next to each cell with title="Add Cell Below" — a clear affordance. The onClick is empty. Clicking it does nothing. Users will click it expecting to insert a cell at a specific position; instead they'll have to use the type buttons at the very bottom of the notebook.

- Auto-save is invisible in both directions. There's no "Last saved X seconds ago" indicator, no dirty state indicator, and no save confirmation. The Save button exists, but a user has no idea if their unsaved changes were captured by the 3-second debounce or if they need to manually save before closing the tab.

- Export button is present but dead. The button is visible, not grayed out, and labeled "Export" — standard affordance for a real feature. Clicking it produces a toast: "Export feature coming soon!" This is a trust break. A placeholder button that looks like a real feature erodes confidence in other buttons.

- Debug cell looks sophisticated, does nothing. Dropdown with 4 options (Current Workflow, Specific Agent, Knowledge Retrieval, Model Performance), two action buttons, a dark console with monospace green text — all theater. No execution path exists.

4. Comparison to Jupyter / ChatGPT

vs. Jupyter Notebook

Capability	Jupyter	JoaLLM Notebook
Shared variable scope across cells	Yes (kernel)	No – each cell isolated
Run cell with Shift+Enter	Yes	No keyboard shortcut
Run all cells in order	Yes	Not implemented
Cell type change after creation	Yes	No
Delete a cell	Yes (D+D)	No delete button in UI
Cell execution counter	Yes (In [3]:)	Tracked in DB, never shown
Error output inline	Yes	"Error: {message}" in pre tag
Interrupt execution	Yes	No
Cell reordering (drag)	Yes	API exists, UI drag unclear
Rich output (matplotlib, tables)	Yes	No – output is always pre-formatted text

Verdict vs Jupyter: Harder, with a fraction of the capability. The surface looks similar (cells, execute, output) but the core mechanism — shared kernel state — is absent. A Jupyter user will be confused and frustrated within 2 minutes.

vs. ChatGPT

Capability	ChatGPT	JoaLLM Notebook
Ask a question, get an answer	Yes	Yes (AI cell)
Context from previous messages	Yes (conversation history)	No – each AI cell is independent
Upload a file and ask about it	Yes (GPT-4)	Yes (Knowledge cell)
Write and run code	Yes (Code Interpreter)	Yes (JS only, server-side)
Visual output (charts, images)	Yes	No
Memory across sessions	Partially	No
One clear input/output flow	Yes	No – user must choose cell type first

Verdict vs ChatGPT: Harder, for less. ChatGPT's conversation thread gives persistent context automatically. Here, each AI cell is a one-shot prompt with no awareness of what came before. To replicate a basic ChatGPT back-and-forth, the user would need to manually copy outputs into subsequent AI cells' content — which is the opposite of what a notebook is for.

The honest comparison: JoaLLM Notebook is currently closer to a form builder where each form field calls a different API — not a notebook.

5. Recommendations to Simplify Usage

Highest impact — fix broken expectations first:

1. Remove or visually disable Chart, Agent, and Debug cells from the UI until they're implemented. Label them "Coming soon" with a lock icon rather than showing a fully styled, interactive UI that does nothing. A grayed-out button that says "Chart (coming soon)" is far better than a working-looking cell that produces no output.
2. Add a delete button to every cell. This is table stakes. A Trash2 icon in the cell toolbar, visible on hover, calling `deleteCell(cell.id)`. Without this, a user who adds a wrong cell type is stuck.
3. Fix the "Add cell below" plus button. It already has the right title attribute, the right icon, the right hover behavior. The `onClick` just needs to call `addCell(type, afterCell.id)`. This one-line fix changes how naturally users can build notebooks.
4. Show save state. Add a "Saved" / "Unsaved changes" indicator in the header next to the Save button. The auto-save debounce is already implemented — just expose its state.

Medium impact — reduce cognitive load:

5. Give AI cells optional conversation context. Add a toggle: "Include previous cells' outputs as context." When on, the AI cell system prompt includes the outputs of all preceding cells. This one change converts the notebook from a collection of isolated prompts into something compositional — without requiring a shared kernel.
6. Make Knowledge cell documents available to AI cells in the same notebook. A global notebook-level document list (already tracked in the DB as `attachedDocuments` per cell) should be accessible to all AI cells as optional RAG context. One checkbox: "Search notebook knowledge base."
7. Rename "Text" button to "Markdown." Or better: rename it "Notes" so it works for both technical and non-technical users.
8. Fix the AI cell icon. Change it from `BarChart3` to `Sparkles` or `MessageSquare`. The current icon creates a visual collision with the Chart cell.

Lower impact — discoverability:

9. Add a `Shift+Enter` shortcut to execute the current cell. This is the single most important keyboard shortcut in any notebook tool. Without it, users must hover and click a play icon for every cell, which destroys flow.
10. Show execution count per cell (`[1]`, `[2]`, etc.) — it's already stored in `executionCount` in the DB schema. Surface it. It tells users execution order and whether a cell has been run.
11. Replace the blank notebook default with a template that demonstrates a working 3-cell flow: Markdown (intro) → Knowledge (upload) → AI (ask about the upload). This shows the product's actual value in the first 60 seconds of use.

Bottom line: The notebook has a credible foundation — the cell architecture, DB schema, and execution backend for Markdown, Code, AI, and Knowledge cells are all real and functional. The usability problems aren't deep engineering issues — they're surface-level: 3 dead cell types polluting the UI, no delete button, no keyboard shortcut, no inter-cell context, and a visual design

that promises Jupyter-level power and delivers something much more limited. Fix the dead cells, add delete, add `Shift+Enter`, and expose save state — that's four changes that would

cut user frustration in half.

Workflow Builder UX (Automation Layer)

Analyze the workflow builder interface.

Focus on:

- How intuitive node creation and connections are
- Clarity of data flow between nodes
- Handling of conditions/branching
- Debugging and error visibility

Output format:

1. How a user builds a workflow step-by-step
2. Whether data flow is understandable
3. Failure/debug experience
4. Key usability gaps

5. Suggestions to improve predictability

Avoid generic UX advice — map to actual interaction patterns.

Codebase:

1. How a User Builds a Workflow Step-by-Step

What the UI implies vs. what actually works:

Step	What user expects	What actually happens
1. Click "New Workflow"	Dialog or inline form	Native browser prompt() dialog – raw OS modal, no styling
2. Drag node from palette	Node drops onto canvas at cursor position	Drag event is set up on palette items; canvas has no onDrop handler – the drag completes but nothing appears
3. Add a node	Node appears, ready to configure	Red + button in bottom-right corner adds a hardcoded LLM node at {x:200, y:200} – this is the only working add path
4. Click node to configure	Properties panel opens	Blue selection ring appears; gear icon's onClick is empty – no configuration panel opens
5. Draw a connection	Drag from output handle to input handle	Handles have no event handlers – they are gray circles that look clickable but do nothing
6. Define branching	Visual fork in the workflow	Two edges with sourceHandle: "truePath"/"falsePath" must be created via API; no UI supports this
7. Reference previous output	Type {{nodeId}} in node config	Correct syntax exists in the executor; no config panel exists to type it into
8. Save	Click "Save"	Works – distinguishes new vs. existing workflows correctly
9. Run	Click "Run"	Works if workflow is saved; polls every 2 seconds
10. See which node failed	Execution history or highlighted node	Toast shows a single error string; no node is highlighted; execution log exists in DB but is never fetched or shown

What a user can actually complete end-to-end today:

- 1. Create a named workflow
- 2. Add LLM nodes via the + button (only LLM type, always at same position)
- 3. Save it
- 4. Run it and receive a pass/fail toast

That is the complete functional surface. Everything beyond that — configuration, connection drawing, branching, variable passing, execution inspection — is scaffolding without wiring.

2. Whether Data Flow Is Understandable

Short answer: No — and not just because the UI is incomplete.

The rendering gap:

Edges are straight <line> elements connecting node centers (x+100, y+25 to x+100, y+25). They're positionally calculated from the node's top-left corner with fixed offsets, not from the actual handle positions. A connection between two nodes draws a line through the middle of both nodes rather than from output handle to input handle. The visual metaphor of "data flows from this port to that port" is not represented.

The label gap:

Edges have no labels. A blue arrow between a Condition node and two downstream nodes looks identical whether it's the "true" branch, the "false" branch, or an unconditional

pass-through. The executor correctly routes on sourceHandle: "truePath"/"falsePath" but the canvas never renders this information.

The data preview gap:

There's no way to hover an edge and see what value flowed through it during execution. The executionLog stored in the DB contains per-node output ("output": { "result": "..."}), but this data is never fetched or displayed on the canvas. After a workflow runs, the canvas looks identical to before it ran.

The interpolation gap:

Variable references use {{nodeId}} syntax — where nodeId is a machine-generated UUID like node-1699123456789. A user would need to know the exact node ID to reference it. There's no UI that shows a node's ID, no autocomplete, no variable picker. The feature exists in the executor and nowhere in the interface.

3. Failure and Debug Experience

What happens when a workflow fails:

1. Run button enters "Running..." state
2. Frontend polls GET /api/workflows/executions/{id} every 2 seconds
3. Polling detects status: 'failed'
4. Red toast appears: "Workflow failed: {error message}" — the error string comes directly from execution.error in the DB
5. Button returns to "Run" state
6. Canvas is unchanged — no node highlighted, no visual change

The execution log problem:

The backend stores a detailed executionLog array with per-node timestamps, status, messages, and output data. A failed execution with node 3 of 7 producing an "API error: invalid key" has that information available in the response. The frontend never requests it, never displays it, and the "Run" flow only calls getExecutionStatus() — which returns the top-level status, not the log.

The execution history problem:

GET /api/workflows/{workflowId}/executions exists. It would show every past run with timestamps, status, and output. There's no panel in the builder that shows this. A user who wants to know "did this workflow succeed yesterday?" has no path through the UI to find out.

The debug cell:

There is a "Debug" node type in the palette. Its configured appearance shows "Mode: Real-time / Logs: Enabled." Its backend execution falls into the default switch case, which returns { result: node.type } — literally just the string "debug". The Debug node does nothing. In a workflow like Input → LLM → Debug → Output, the Debug node produces the text "debug" as its result.

The settings panel checkbox problem:

The workflow settings panel has an "Auto-save results" checkbox and a "Parallel execution" checkbox. Neither checkbox has an onChange handler. Both Settings fields are read from workflow.settings.autoSave and workflow.settings.parallelExecution respectively, but the Save Changes button inside the panel doesn't include these values in the update payload. They are rendered but never persisted.

4. Key Usability Gaps

Ranked by how fundamentally they break the workflow-building task:

Blockers (feature is entirely non-functional):

1. Drag-and-drop from node palette does nothing. The primary method for adding nodes — the one the entire left panel is built around — has no drop handler on the canvas. The only working method is a red + button in the corner that always adds an LLM node at a fixed position. Users who try to drag nodes will assume the feature is broken.
2. Node configuration is not possible. Every node shows hardcoded summary text ("Model: GPT-4", "Temperature: 0.7"). The gear icon that would open a config panel has an empty onClick. A user cannot change the model, the prompt template, the condition logic, or any other node setting through the UI.
3. Connections cannot be drawn by the user. Handles are visual. Neither handle responds to mousedown, click, or drag events. Edges must be created by calling WorkflowContext.addEdge() directly — accessible only to developers, not users.

Critical gaps (feature works but critical information is missing):

4. No execution history panel. The backend stores every execution with full logs. The UI exposes none of it. After a run, there's no record visible to the user.
5. No per-node error visibility. The toast shows "Workflow failed: API error" without identifying which of the 7 nodes failed. The execution log has this information. A user cannot debug a failed workflow without database access.
6. Conditional branches have no visual labels. True/false paths look identical to unconditional edges. A workflow with branching is visually indistinguishable from one without.

Notable gaps (degraded experience):

7. Native browser prompt() for workflow naming. First interaction with the builder is an unstyled OS dialog box. In a product where every other UI element is polished, this stands out sharply.
8. No autosave. Nodes, edges, and workflow changes are only persisted on manual "Save" click. A browser refresh or tab close loses all unsaved work silently.
9. No node deletion confirmation. Clicking the X icon on a node calls deleteNode(id) immediately. On a complex workflow with configured nodes (once configuration works), this is destructive with no undo.

10. Edge rendering connects node centers, not handles. Lines pass through node bodies rather than emerging from ports, making the graph look like a diagram of collisions rather than a flow.

5. Suggestions to Improve Predictability

These are mapped to the specific implementation gaps above, not generic advice:

Fix the interaction model before adding features:

1. Implement the canvas onDrop handler immediately.

NodePalette.tsx already does `e.dataTransfer.setData('nodeType', nodeType)`. The canvas needs:

```
onDrop={(e) => {  
  const type = e.dataTransfer.getData('nodeType');  
  const rect = canvasRef.current.getBoundingClientRect();  
  addNode({ type, position: { x: e.clientX - rect.left, y: e.clientY - rect.top } });  
}}  
onDragOver={(e) => e.preventDefault()}
```

This is a 5-line fix that makes the primary interaction work.

2. Connect handle mousedown events to draw edges.

When a user mousedown on a bottom handle, start tracking a "pending edge" in canvas state. On mouseup over a top handle of another node, call `addEdge(sourceId, targetId)`. Render the pending edge as a dashed line following the cursor. This is the standard interaction pattern for every node graph tool and what users will attempt on first use.

3. Open a config panel when the gear icon is clicked.

Each node type already has a predictable data shape (LLM needs model + prompt, Condition needs value + operator + compareTo). A right-side config panel that renders form fields based on node.type — and calls `updateNode(id, { data: {...} })` on change — completes the build loop. Without this, the workflow builder has no purpose.

4. Add edge labels for condition branches.

When an edge's sourceHandle is "truePath" or "falsePath", render the label inline on the SVG line:

```
<text x={(x1+x2)/2} y={(y1+y2)/2} fill="#16a34a" fontSize="10">  
  {edge.sourceHandle === 'truePath' ? 'TRUE' : 'FALSE'}  
</text>
```

Two lines of SVG. Eliminates the entire "which branch is which" ambiguity.

5. Show execution log on the canvas after a run.

After execution, fetch the full execution object (not just status). For each node in the log, add a visual state: green checkmark for completed, red X for failed, gray for not reached. Show the node's output value as a tooltip or inline badge. This turns the canvas from a static diagram into an execution trace — the actual value of a visual workflow tool.

6. Replace `prompt()` with an inline title field.

The notebook already does this correctly: an editable `<input>` in the header, editable on click, saved inline. Apply the same pattern to workflow naming.

7. Implement autosave with a dirty state indicator.

The notebook has a 3-second debounced save. Copy that pattern here. Add a small "Unsaved changes" badge next to the workflow name that disappears after save. The current pattern (manual save required, no dirty indicator) means users will lose work.

Summary assessment: The workflow builder has a correct architecture — the executor's `{{variable}}` interpolation, the `truePath/falsePath` branching logic, the execution log schema, and the polling-based run flow are all sound engineering. The UI scaffold is also present — node palette, canvas, header, settings panel, node component. But the three interactions that constitute

"building a workflow" (add a node, configure a node, connect two nodes) are all non-functional. The product has a complete back and a complete front with nothing connecting them. The fix is not a redesign — it's wiring up event handlers that are already stubbed out.

Navigation & Mental Model

HI: Evaluate the overall navigation and structure of the product.

Focus on:

- Whether the product feels like a tool, platform, or workspace
- How easily users can find key features
- Consistency in layout and terminology

Output format:

1. Mental model required to use the product
2. Navigation clarity (clear / confusing with reasons)
3. Areas where users may feel lost
4. Suggested simplifications

Highlight mismatches between UI structure and user expectations.

Codebase:

1. Mental Model Required to Use This Product

A user needs to hold three simultaneous mental models to navigate comfortably — and none of them are made explicit:

Model 1 — "Tool collection" (what the sidebar implies)

Seven separate tools exist (Chat, Knowledge Base, Notebook, Workflow, Farm, Docs, Welcome). Each is a distinct feature with its own page. You pick the tool you need.

Model 2 — "Workspace with a shared brain" (what the landing page promises)

"Upload. Query. Automate." implies these tools work together — upload a doc, query it in chat, automate the workflow. But in practice they don't share state. A document uploaded in Knowledge Base isn't accessible in Notebook. A model selected in Chat doesn't persist context to Workflow. The mental model the landing page sells is not the one the product delivers.

Model 3 — "Role-based platform" (what the user dropdown suggests)

Switching between Developer / Data Analyst / Business User / Casual User implies the product changes. The roles have different module access lists in code (developer gets 8 modules, casual gets 2). But no route-level enforcement exists in App.tsx — every authenticated user can access every URL regardless of role. The role switcher changes UI labels but not access. A user who switches to "Business User" expecting a simplified view will see the same interface.

The core problem: There is no single coherent mental model. The product looks like a platform (side-by-side tools), works like a collection of disconnected apps, and is described as a workspace. Users who expect any one of these three will be partially right and partially confused.

2. Navigation Clarity

Rating: Confusing — with specific reasons

What works

- URL-based routing is clean and consistent (/chat, /rag-search, /notebook, /workflow, /farm)
- Active state in the sidebar correctly highlights the current section

- Sidebar auto-closes on mobile after navigation — prevents the common trap of navigating behind an open overlay
- The back-to-hub pattern in Chat/Notebook/Workflow sidebars gives a clear escape hatch

What doesn't work

Problem 1 — Two navigation systems competing.

The header has a row of icon buttons (Chat, Knowledge Base, Settings) that partially duplicate the sidebar. The sidebar has 7 nav items. The header quick actions only show 3 of them (Chat, Knowledge Base, Settings). A user clicking around will discover they can reach Chat from 3 different places: sidebar nav item, header icon, quick-start prompts. This multiplies navigation options without adding clarity.

Problem 2 — "Chat Assistant" vs "AI Assistant" — the product can't agree on what to call its main feature.

- Sidebar label: "Chat Assistant"
- Header ViewMode title: "AI Assistant"
- Welcome page card title: "Intelligent Chat"
- URL path: /chat
- Four different names for one feature. A user looking for where they were 5 minutes ago has no reliable term to anchor to.

Problem 3 — Sidebar context changes unpredictably.

When on /chat, /notebook, or /workflow, the sidebar switches from the main "LLM Hub" nav to a feature-specific panel. When on /rag-search, /farm, or /docs, it stays on the LLM Hub nav. There's no rule the user can learn — some views replace the sidebar, others don't. The experience of pressing "Back to LLM Hub" only exists for some pages.

Problem 4 — The model selector disappears on mobile.

Model selection is a core differentiator of the product. On mobile (hidden md:block), the entire model selector is hidden with no alternative access. Users on mobile have no way to change the AI model. This isn't a minor responsive issue — it removes the product's primary value-add on the most common device type.

Problem 5 — Settings, Knowledge Manager, and Bookmarks are modals, not pages.

These three features appear as overlays without their own URLs. They can't be bookmarked, linked, or navigated to directly. Settings specifically is a feature users return to repeatedly — it should be /settings. Using a modal means it doesn't appear in browser history, can't be deep-linked, and is inaccessible from the sidebar on mobile.

Problem 6 — Header quick action icons have no active state.

The 3 header icon buttons (Chat, Knowledge Base, Settings) look identical whether the user is on that page or not. If you're on the Knowledge Base page, the Knowledge Base icon in the header should look selected. It doesn't. The sidebar shows correct active state; the header doesn't.

3. Where Users Will Feel Lost

Lost moment 1: "Where are my documents?"

A user uploads documents in the Knowledge Base (/rag-search, tab 3: Knowledge Manager). They then go to Notebook and add a Knowledge cell expecting those documents to be there. They're not — Knowledge cells have their own per-cell document scope. The user searches the interface for a way to connect the two. There is none.

Lost moment 2: "What does switching roles do?"

The user dropdown shows 4 roles with distinct colors and labels. A "Business User" switching to "Developer" sees... the same 7 sidebar items, the same pages, the same features. The role system's intended UX (simplified view for business users, full access for developers) is built in the context system but not enforced in routing. Users switch roles expecting something to change and experience nothing visible.

Lost moment 3: "What is JoaLLM Farm?"

Every other nav item has a recognizable name: Chat, Knowledge Base, Notebook, Workflow. "Farm" has no analog in any tool the user has seen before. The sidebar description says "Explore all available AI models" but that doesn't explain why it's called Farm or what "exploring models" achieves. After visiting, users find a model browser — which they already interacted with in the header model selector. Two places to browse models, one of which is named something opaque.

Lost moment 4: "How do I get back?"

From the Chat sidebar, there's a "Back to LLM Hub" button. From Notebook, same. From RAG Search, there's no back button because the sidebar didn't switch — you're already on LLM Hub. A user who navigated to RAG Search from Chat will look for a "back" affordance and find the sidebar looks the same as before they navigated. The back button is contextual, not consistent.

Lost moment 5: "Is Documentation a real feature?"

The Documentation nav item (/docs) appears with equal visual weight to Chat, Knowledge Base, and Notebook. Inside, it shows a sidebar with status indicators: green "Completed", yellow "Work in Progress", blue "Planned." The sidebar of the Documentation section describes what the platform is building. A user who opens it looking for help with the product will instead find a product roadmap. Help and roadmap are in the same place with the same visual treatment.

Lost moment 6: "Where is the Notebook icon in the header?"

The header quick actions include Chat (MessageSquare) and Knowledge Base (Database) but not Notebook or Workflow. There's no consistent logic for why Chat and Knowledge Base were promoted to the header while Notebook and Workflow weren't. The selection feels arbitrary.

4. Suggested Simplifications

Structural — eliminate competing navigation systems:

1. Remove the header quick action icons or make them a complete set. Either show all 5 main tools in the header, or show none and let the sidebar be the sole navigation. Currently 3 tools appear in the header and 7 in the sidebar — the partial overlap creates ambiguity without adding speed.
2. Make the sidebar context consistent. Either all pages switch the sidebar to a feature-specific panel, or none do. The current hybrid (Chat/Notebook/Workflow switch; RAG/Farm/Docs don't) has no learnable rule. Simplest fix: never switch the sidebar — always show the main nav with the active item highlighted.
3. Give Settings its own route (/settings). Move it from a modal to a page. Add it to the sidebar below the main feature nav. This makes it findable, bookmarkable, and consistent with how every other feature works.

Naming — remove ambiguity:

4. Pick one name for the chat feature and use it everywhere. Recommendation: "Chat" — one word, already the URL, already what users call it. Replace "Chat Assistant," "AI Assistant," and "Intelligent Chat" with just "Chat."
5. Rename "JoaLLM Farm" to "Models." One word. Matches the URL convention (currently /farm — change to /models). Eliminates the only opaque term in the nav.
6. Rename "Documentation" to "Docs & Roadmap" or move the roadmap content out of the Docs page. A user looking for help should find help, not a development status board.

Role system — make it real or remove it:

7. Either enforce role-based routing or remove the role switcher. The current state is the worst of both: the UI implies access changes, but it doesn't. If enforcing roles is out of scope, remove the role switcher from the user menu entirely. The role indicator at the bottom of the sidebar (colored dot + role name) can stay as a display-only label if needed for personalization.

Mobile — restore the model selector:

8. Add a model selector to the mobile sidebar. The sidebar already has space and the pattern (collapsible section with a list) exists. Add a "Model" section to the sidebar's LLM Hub panel that expands to show model options. This restores the product's core differentiator on mobile.

Cross-feature — surface the connections:

9. Add a "Use in..." shortcut from Knowledge Base documents. When a document is indexed, show: "Open in Chat →" and "Open in Notebook →" links that deep-link to those features with the document pre-selected. This makes the cross-tool connection visible without requiring a full shared-state architecture.

The single biggest structural issue: The product is six apps wrapped in one shell, presented as a unified platform. The navigation reflects this — each tool has its own URL, its own sidebar state, its own empty state, and no handoffs to adjacent tools. Until the tools share at least one piece of state (documents, conversation history, or model selection), no amount of navigation restructuring will make it feel like one product. The navigation is currently honest — it exposes the seams between six disconnected tools. The fix isn't purely navigation; it's connection.

Model Selection & AI Abstraction UX

HI:Analyze how LLM/model selection is exposed to users.

Focus on:

- Whether users need to understand providers
- Clarity of cost, performance, and trade-offs
- Default vs advanced usage paths

Output format:

1. How model selection is presented
2. Whether it adds value or complexity
3. Ideal user path (default vs advanced)
4. Recommendations to simplify decision-making

Goal: minimize cognitive load for non-technical users.

Codebase:

1. How Model Selection Is Presented

The product exposes model selection at three separate layers, each with a different level of detail:

Layer 1 — Header selector (always visible, desktop only)

A persistent button showing: ● Llama 3.3 70B Versatile (Groq) [Groq badge] ▼. Always visible on desktop, completely hidden on mobile with no alternative.

Opening it reveals a dropdown with:

- A text search input
- A provider filter dropdown
- A "Trending" toggle and a "Favorites" toggle
- A "Quick Access" section (recent + favorites, max 4)
- An "All Models" section listing every model

Each model card shows simultaneously: name, description, provider badge, speed icon + label, \$0.59/1M in + \$0.79/1M out pricing, max token count formatted as "32K tokens", and up to 3 capability tags. That is 7 data dimensions per card, in a scrollable list of 48 models.

Layer 2 — JoaLLM Farm (/farm)

A full-page model browser with 5 filter dropdowns (search, provider, capability, speed, quality), a paginated grid (12/24/48 per page), and model cards that additionally show the raw model ID in a monospace code block and quality stars. This is more information than the selector, on a dedicated page.

Layer 3 — Model Parameters panel

A collapsible panel with 3 presets (Creative / Balanced / Precise) and 5 sliders (temperature, maxTokens, topP, frequencyPenalty, presencePenalty). Parameters are globally scoped — changing temperature in settings changes it everywhere, for every feature.

Defaults:

- Selected at startup: Llama 3.3 70B Versatile (Groq) — a model name a non-technical user cannot evaluate
- Temperature: 0.7, maxTokens: 2048, others at 0
- No auto-selection based on use case; the same default applies whether the user is writing an email or debugging code

2. Whether It Adds Value or Complexity

For technical users: genuine value.

A developer who knows the difference between llama-3.1-8b-instant (560 tok/s, \$0.05/1M) and claude-3-sonnet (\$3/\$15 per 1M) has real reason to switch. The pricing format, speed indicators, capability tags, and model IDs for API reference are all appropriate.

For non-technical users: pure complexity, no value.

The problems are specific:

Problem 1 — Cost format requires inference.

"\$0.59/1M in + \$0.79/1M out" is meaningless to a user who doesn't know what "1M tokens" represents in conversational terms. A typical chat message is ~200–400 tokens. At \$0.59/1M input, a conversation costs fractions of a cent. The format is technically accurate but practically uninterpretable without a reference frame.

Problem 2 — 48 models with provider names the user doesn't recognize.

The list includes: Llama 3.3 70B, Llama 4 Scout 17B, Llama 4 Maverick 17B, GPT-OSS 120B, Groq Compound, Groq Compound Mini, Qwen 3 32B, Kimi K2, Allam 2 7B, Whisper Large V3, PlayAI TTS Arabic, Llama Guard 4 12B, Llama Prompt Guard 2 22M. These are presented in an undifferentiated list to a user who wants to "ask a question about a document." The presence of speech-to-text

(Whisper), TTS (PlayAI), and safety/guard models (Llama Guard, Prompt Guard) in the same list as chat models creates false options — a user who selects PlayAI TTS for a chat query will get an unintelligible result with no error explaining why.

Problem 3 — Speed labels (fast/medium/slow) without absolute benchmarks.

"Fast" means different things depending on context. Llama 3.1 8B at 560 tok/s and GPT-4 at ~40 tok/s are both labeled by relative tier. A user who picks "fast" because they want snappy replies doesn't know whether that means 1-second or 10-second responses.

Problem 4 — Quality is tracked but never displayed in the primary selector.
The quality: 'high' | 'medium' | 'low' field exists on every model. The Farm page shows quality stars. The header selector never displays it. The one dimension most relevant to a non-technical user's decision ("which model is better?") is absent from the primary interaction point.

Problem 5 — Model selection is global but the implications aren't.
Changing the model in the header affects Chat, RAG Chat, and (implicitly) Notebook simultaneously. A user who switches to a small fast model for quick chat responses doesn't know they've also degraded their RAG responses. There's no per-feature model assignment and no indication of the global scope.

Problem 6 — Non-chat models are in the chat selector.
Whisper Large V3 (speech-to-text), PlayAI TTS (text-to-speech), PlayAI TTS Arabic, Llama Guard 4 12B (safety classifier), Llama Prompt Guard 2 22M/86M (prompt injection detection) — these are all selectable from the chat model dropdown. They are not chat models. Selecting one produces either no output or nonsense output. There's no filtering or labeling that separates inference models from specialized models.

3. Ideal User Path: Default vs. Advanced

What the current product assumes:
Every user is a technical user who wants to evaluate and compare models. The default path is: see 48 models → decide based on provider, speed, cost, tokens, and capabilities → select → optionally tune 5 parameters.

What a realistic user split looks like:

Step	What user expects	What actually happens
1. Click "New Workflow"	Dialog or inline form	Native browser prompt() dialog – raw OS modal, no styling
2. Drag node from palette	Node drops onto canvas at cursor position	Drag event is set up on palette items; canvas has no onDrop handler – the drag completes but nothing appears
3. Add a node	Node appears, ready to configure	Red + button in bottom-right corner adds a hardcoded LLM node at {x:200, y:200} – this is the only working add path
4. Click node to configure	Properties panel opens	Blue selection ring appears; gear icon's onClick is empty – no configuration panel opens
5. Draw a connection	Drag from output handle to input handle	Handles have no event handlers – they are gray circles that look clickable but do nothing
6. Define branching	Visual fork in the workflow	Two edges with sourceHandle: "truePath"/"falsePath" must be created via API; no UI supports this
7. Reference previous output	Type {{nodeId}} in node config	Correct syntax exists in the executor; no config panel exists to type it into
8. Save	Click "Save"	Works – distinguishes new vs. existing workflows correctly
9. Run	Click "Run"	Works if workflow is saved; polls every 2 seconds
10. See which node failed	Execution history or highlighted node	Toast shows a single error string; no node is highlighted; execution log exists in DB but is never fetched or shown

- The path that would serve 80% of users:
1. Open the product
 2. Default model is already selected (something good for general use, not llama-3.3-70b-versatile by ID)
 3. Use the product without touching model selection at all

4. Optionally change to "Creative," "Fast," or "Most Accurate" — human-readable categories

The path that currently exists for those 80%:

1. Open the product
2. See ● Llama 3.3 70B Versatile (Groq) in the header
3. Wonder what that means
4. Click it and see 48 models with token pricing
5. Either pick randomly or leave the default
6. Proceed — and have no way to know if that was a good choice

4. Recommendations to Simplify Decision-Making

These are tied to the specific implementation, not generic advice:

1. Rename the default model's display name.

"Llama 3.3 70B Versatile (Groq)" → "Llama 3.3 (Recommended)". The closed-state button is a user's persistent reference to "what AI am I using." The current name requires knowing what "3.3," "70B," and "Groq" mean. A parenthetical like "(Recommended)" or "(Best for general use)" costs nothing to add and answers the first question every non-technical user has.

2. Filter out non-chat models from the chat selector.

Add a chatCompatible: boolean field (or type: 'chat' | 'speech' | 'embedding' | 'safety') to the model schema. In the header selector and RAG chat, filter to type === 'chat' only. Whisper, PlayAI TTS, Llama Guard, and Prompt Guard models should not appear in a chat context. This reduces the list from 48 to ~15 without removing any legitimate options.

3. Replace the closed-state selector display.

Currently: ● Llama 3.3 70B Versatile (Groq) — meaningless to most users.

Proposed: ● Llama 3.3 · Fast · General Purpose — name + two plain-English attributes.

Implementation: add shortDescription: string to LLMModel type (e.g., "Fast · General Purpose", "Powerful · High Quality", "Free · Local"). Show this instead of provider badge in the closed state.

4. Add a three-option simple selector above the model list.

Before the full dropdown list, add three clearly labeled smart defaults:

Best match for this task	
☐ Fast & Free ☐ Balanced ☐ Best Quality	

[Advanced: choose from all models ▼]

Map these to specific models: Fast & Free → llama-3.1-8b-instant, Balanced → current default, Best Quality → claude-3-sonnet or gpt-4-turbo. The full 48-model list stays accessible under "Advanced" but isn't the first thing a user sees.

5. Change cost display to a relative format for non-technical users.

"\$0.59/1M in + \$0.79/1M out" → show the cost label only to users who've opted into advanced mode. For default view, show a cost tier: ● Free, ●● Low cost, ●●● Premium. The raw API pricing is meaningful to developers billing clients; it's noise to everyone else.

6. Make the quality field visible in the primary selector.

Move the quality indicator (currently only in Farm) into the header selector card. A non-technical user's primary decision axis is "which is better?" The selector currently shows speed and cost but not quality. Swap the priority: quality first, then speed, then cost — in that order.

7. Scope model parameters per feature, or at minimum add a scope indicator.

The parameters panel applies globally but has no label indicating this. Add a banner: "These settings apply to all AI features." Better: allow per-feature overrides with a "Use different settings for this feature" toggle. At minimum, warn the user when they change parameters that it affects the entire platform.

8. Remove the JoaLLM Farm from the primary navigation.

The Farm duplicates the model selector with more data. It's useful for evaluation, not for daily use. Moving it out of the main nav (or making it a sub-section of Settings) removes one confusing nav item and clarifies that it's a reference tool, not a feature. Alternatively, make the Farm the landing page for "Advanced" mode in the model selector — so clicking "choose from all

models" goes to the Farm rather than a scrollable dropdown.

The core tension: The product is built around model diversity as a feature ("20+ AI Models, 4 LLM Providers" in the landing stats). This creates pressure to expose all models prominently. But model choice is only valuable if users can make an informed decision — and 48 models with token pricing doesn't enable decisions, it defers them. The fix is a two-tier system: a curated

3–5 model "simple" selection that covers 90% of use cases, and a full catalog for users who want it. Both can coexist; currently only the full catalog exists.

Trust, Transparency & Safety UX

Evaluate how the UI builds user trust in AI outputs.

Focus on:

- Visibility of sources used in responses
- Indicators of confidence or uncertainty
- Data privacy cues (security, access control)

Output format:

1. Trust signals present in UI
2. Missing transparency elements
3. Risk areas (where user may distrust output)
4. Suggestions to improve trust

Assume enterprise users (legal/finance mindset).

Error Handling & Failure UX

Analyze how the system handles errors and failures.

Focus on:

- What happens when actions fail (upload, query, workflow)
- Error messaging clarity
- Recovery options for users

Output format:

1. Types of errors visible to users
2. Quality of error messages
3. Ease of recovery
4. UX gaps that may frustrate users

Prioritize real-world usability over technical correctness.

Cognitive Load & Decision Fatigue

Evaluate the cognitive load imposed on a new user.

Focus on:

- Number of decisions required in first 5 minutes
- Complexity of terminology
- Overloaded UI elements

Output format:

1. Number of key decisions early on
2. Areas of overload
3. Simplifications possible
4. What can be automated or hidden

Goal: reduce decision-making by at least 50%.

End-to-End UX Judgment (Final Synthesis)

Based on the full UI/UX:

1. What is the primary job this product helps users accomplish?
2. How easy is it for a new user to achieve that job?
3. What is the biggest UX blocker today?
4. What is the most powerful UX element?
5. Is this product simple, powerful, or confusing? Why?

Be opinionated and specific.
Avoid generic UX language.

BONUS: Live User Testing Prompt

You have 15 minutes.

Goal:
Upload a document and get useful insights from it.

Rules:

- Don't ask for help
- Think out loud

After:

1. What confused you?
2. What felt powerful?
3. Where did you get stuck?
4. Would you use this again?